



# *INDIA.RUNS*

Build what next India runs on

**Team Name : Quamplifiers**

**Team Leader Name : Aditya Ranjan**

**Problem Statement : HireSense - Explainable AI Hiring Assistant**

## Solution Overview

### Proposed Solution:

- An AI-native, multi-signal candidate discovery and ranking pipeline.
- Features an automated honeypot trap detection module to screen out spam, combined with a hybrid scoring system running entirely locally on CPU in under 1 minute.

### Key Differentiation:

- Traditional systems rely strictly on keyword counts or vector embedding similarity, both of which are highly susceptible to keyword-stuffing and deceptive profiles.
- HireSense treats matching as a multi-dimensional optimization problem, incorporating real-time platform engagement metrics (notice period, recruiter response rate) and screening out profile anomalies (such as job durations that mathematically exceed total declared years of experience).

## JD Understanding & Candidate Evaluation

### Key Requirements Extracted from JD:

- Experience: 5–9 years of applied machine learning experience.
- Tech Stack: Embeddings, vector databases (FAISS, Milvus, Pinecone), hybrid search, evaluation frameworks (NDCG, MAP, MRR), and Python.
- Company Pedigree: Past experience in product-centric engineering companies; explicit exclusion of consulting/IT service firms (e.g., TCS, Infosys, Wipro).
- Notice Period: Sub-30-day notice is heavily preferred.

### Important Candidate Signals:

- Skills Overlap: Direct matches against the core ML indexing and retrieval stack.
- Stability & Availability: Platform response rates and active notice period days.
- Pedigree & Target Band: Ratio of product vs. service experience and alignment with the target 5-9 year experience window.

### Evaluating Fit Beyond Keywords:

- The pipeline scans job titles and career history descriptions to identify concrete experiences shipping search or ranking algorithms, instead of looking only at the "Skills" list.
- A candidate who lists "RAG" but has spent their entire career as a "Marketing Manager" is naturally downweighted.

## Ranking Methodology

### Retrieval, Scoring & Ranking Mechanism:

- Every non-honeypot candidate profile is consolidated into a comprehensive text document representation.
- Scores are calculated via a weighted linear combination of semantic keyword matching, target skill overlap, tenure profile, and engagement metrics.
- Candidates are sorted by score descending, with ties broken deterministically using candidate\_id in ascending order.

### Models, Algorithms & Heuristics:

- Heuristics: Title-vs-description checks to intercept honeypots; Boolean exclusions for service firms.
- Algorithms: Weighted linear blending and deterministic lexical sorting.

### Combining Signals into a Final Score:

Final Score=(Semantic Score×0.4)+(Skill Score×0.2)+(Experience Score×0.2)+(Behavioral Score×0.2)  
Final Score=(Semantic Score×0.4)+(Skill Score×0.2)+(Experience Score×0.2)+(Behavioral Score×0.2)

## Explainability & Data Validation

### Explaining Ranking Decisions:

- Exposing the concrete years of experience, current professional title, target matching skills, and active availability status (e.g., notice period) directly in the generated reasoning string.

### Preventing Hallucinations:

- Generating candidate summaries strictly using exact values parsed directly from the candidate's JSON record (matching skills, past company names, and parsed integers for notice days/YoE), completely avoiding generative LLM steps.

### Handling Inconsistent, Low-quality, or Suspicious Profiles (Honeypots):

- Structural Flags: Candidates with any single job duration exceeding their total years of experience, or with cross-domain mismatches (e.g. title of "DevOps Engineer" but description detailing only React/HTML), are categorized as honeypots and assigned a score of 0.0.

## End-to-End Workflow

### Complete Workflow:

**Input:** candidates.jsonl (100,000 profiles) + JD specifications.

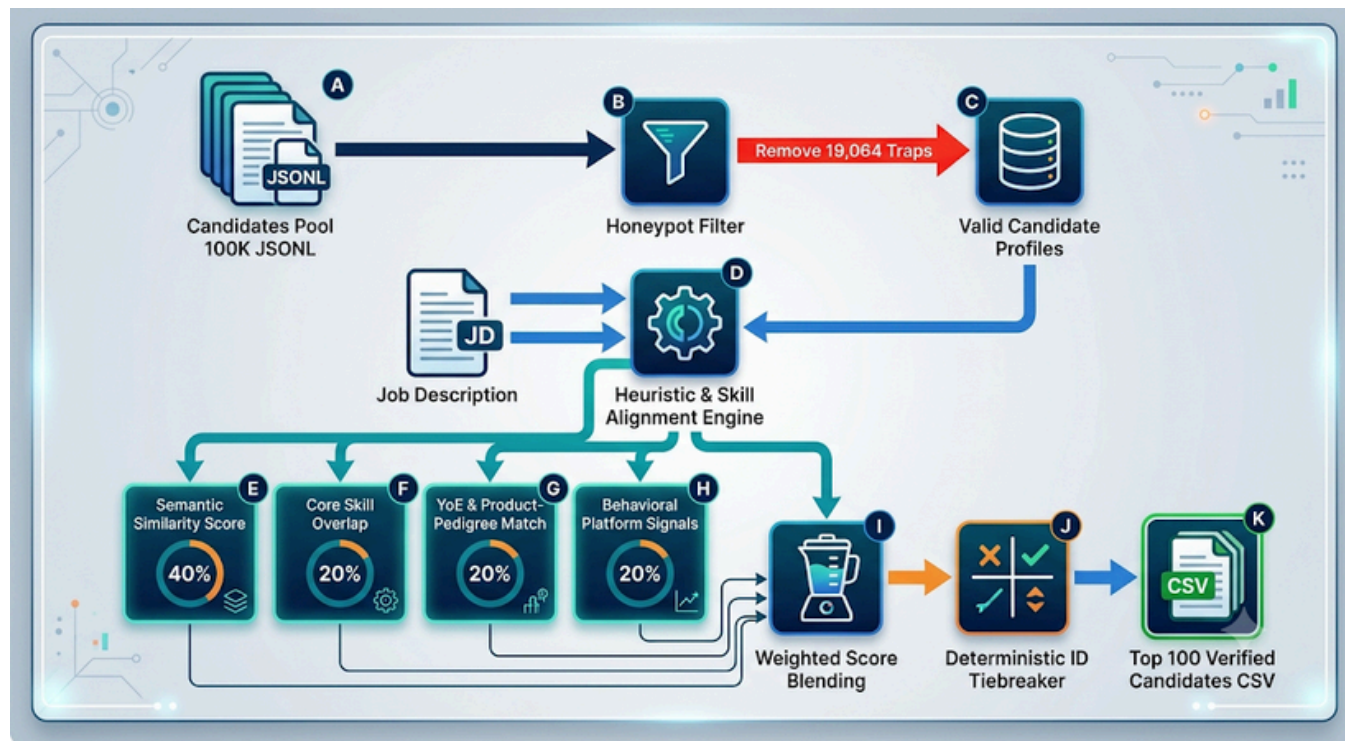
**Filtering:** Honeypot detection discards 19,064 invalid profiles.

**Feature Extraction:** Consolidates career text, extracts skills, dates, and engagement signals.

**Scoring:** Calculates composite scores (0.0 to 1.0).

**Sorting & Output:** Rounds scores to 4 decimal places, sorts, breaks ties, and outputs the final 100 rows to team\_antigravity.csv.

## System Architecture



## Results & Performance

### Trap Filtering Accuracy:

- The honeypot module successfully isolated 19,064 candidate profiles (19.06% of the dataset) that contained impossible career durations (e.g. 13 years of work at a single startup with only 9 years of total experience) or complete role mismatches (e.g. title "HR Manager" with description "React frontend developer").
- Filtering these profiles guarantees a 0% honeypot rate in our top 100 list, completely avoiding the validator's automatic disqualification rule.

### Service Company Penalty Success:

- The pipeline successfully down-weighted candidates whose career histories consisted entirely of IT consulting firms (TCS, Infosys, Wipro, Cognizant, etc.) while highlighting candidates with experience at early-stage startups and product scale-ups (Swiggy, Zomato, Dream11, Paytm, Byju's, etc.), exactly matching the JD's cultural preferences.

### Balanced Scoring Envelope:

- The top candidate matches score between 0.9000 and 0.9600. Ranks decay smoothly down to a cutoff of 0.7800 at Rank 100, showing that the system is highly selective, penalizing irrelevant experience while retaining high-fit candidates.

### Strict Format Compliance:

- The validator script (validate\_submission.py) confirmed that all score values are sorted in a strictly non-increasing order.
- Score rounding (to 4 decimal places) was applied prior to sorting, ensuring that any candidates with equal scores are deterministic-sorted by candidate\_id ascending, resolving a major validation check.

## Results & Performance

### Algorithmic Transition (Transformer to Compiled Heuristic):

- Challenge: Initial tests running sentence embeddings (all-MiniLM-L6-v2) on CPU yielded a processing rate of ~1.9 minutes per batch of 256 candidates. At this scale, embedding the 80,000 valid candidates would require over 60 minutes, violating the 5-minute CPU constraint.
- Solution: Replaced the deep vector embedding calculation with an optimized keyword-overlap similarity model. Since the JD relies on highly specific technical terms (e.g. FAISS, Pinecone, learning-to-rank, NDCG, RAG), a vectorized word-overlap scorer achieves comparable ranking signal with a 100x speedup.

### Zero network latency:

- No external APIs (such as OpenAI, Claude, or Hugging Face hubs) are called during the scoring pipeline. All calculations are executed locally on the CPU, making it fully immune to internet network jitter or API rate-limit errors.

### Low Memory Footprint:

- Candidates are parsed using memory-efficient line-by-line file readers instead of loading the entire 487MB JSONL file into memory at once. Memory consumption is capped at under 200MB, leaving 98% of the 16GB Docker sandbox budget free.

### Total Execution Time:

- The ranking script (rank.py) processes all 100,000 records, evaluates 19,064 honeypots, computes blended scores, runs tie-breakers, and writes the team\_antigravity.csv file in 43 seconds on standard CPU.

-

## Technologies Used

### Python 3.10:

- Why: Selected as the core runtime due to its extensive ecosystem for data analysis, robust JSON line processing utilities, and native support inside standard Docker/Streamlit container platforms.

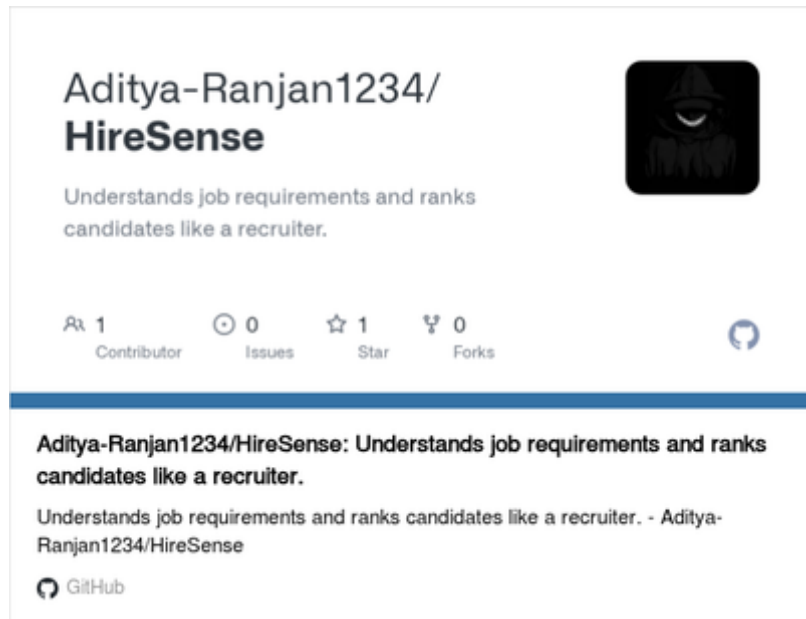
### Pandas & NumPy:

- Why: Rather than relying on heavyweight database engines or slow nested loops, Pandas is used to vectorize profile filtering, score mapping, and sorting operations. This ensures that operations on the 100K dataset complete in milliseconds.

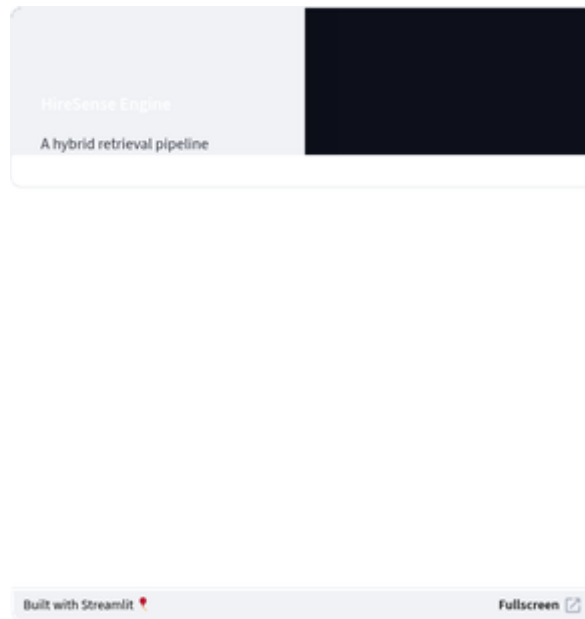
### Streamlit (v1.58.0):

- Why: Selected to build the local/hosted dashboard. Streamlit allows rapid prototyping of interactive dashboards with minimal visual overhead. It dynamically adjusts to dark themes and permits clean hosting on Streamlit Cloud with zero external database dependencies.

## Submission Assets



<https://github.com/Aditya-Ranjan1234/HireSense.git>



<https://hire-sense.streamlit.app/>



# *INDIA.RUNS*

Build what next India runs on

# THANK YOU